

TalentFlow ATS - Technical Documentation

Core Java 21 Recruitment & Applicant Tracking System | Concepts Mapping

1. Executive Architecture Overview

TalentFlow is an enterprise-grade Recruitment & Applicant Tracking System written in **Pure Core Java 21** without external frameworks (No Spring Boot, JavaFX, Swing, Maven, or Gradle). The system implements a clean **Layered (N-Tier) Architecture**, dynamic **Max-ID Auto-Incrementing**, **Permanent Disk Storage Persistence**, and **Role-Based Access Control (RBAC)**.



2. Complete File & Concept Mapping

Model Layer (src/model/)

User.java

- **Abstraction:** Declared as an abstract class User. Serves as the base model for system actors and cannot be instantiated.
- **Abstract Method:** Declares public abstract void displayDetails() forcing child classes to implement concrete display logic.
- **Encapsulation:** Private fields (id, name, email, phone, password) protected behind public getters and setters.

Candidate.java

- **Inheritance (extends):** Extends User to inherit common profile fields.
- **Polymorphism (Overriding):** Overrides displayDetails() and toString() to display candidate profile fields (skill, experience, status, resumeSummary).

Recruiter.java

- **Inheritance (extends):** Extends User to inherit common user fields.
- **Polymorphism (Overriding):** Overrides displayDetails() and toString() for recruiter attributes (company).

Job.java

- **Encapsulation:** Protects job posting properties (jobId, title, company, location, salaryRange, requiredSkill).
- **Constructor Overloading:** Supports default vs custom salary range initialization.

Application.java

- **Data Model:** Links Candidate ID to Job ID with status workflow tracking (Applied, Interview, Selected, Rejected, Withdrawn).

Repository Layer (src/repository/)

- **CandidateRepository.java:** DAO pattern using ArrayList<Candidate>. Uses static {} to read disk data from data/candidates.txt. Implements dynamic max-ID auto-increment getNextId() (starting at 501) and unique email verification.
- **RecruiterRepository.java & JobRepository.java:** Static disk persistence with dynamic max-ID auto-increment (Recruiter ID from 1, Job ID from 101).
- **ApplicationRepository.java:** Manages application records with dynamic max-ID (from 1001) and cascade deletion integrity (purging orphaned applications when candidate or job is deleted).

Service Layer (src/service/)

- **CandidateService.java & JobService.java:** Implements method overloading for ID vs String searches (e.g. searchCandidate(int) vs searchCandidate(String)). Implements privacy view filtering (hiding required skills in public view).
- **ApplicationService.java:** Automated Skill Matching Engine comparing candidate skills against job requirements using string matching algorithms. Renders dual-interface views for Candidate vs Recruiter portals.

Utility & Presentation Layers (src/utill/ & src/menu/)

- **Validation.java:** Regular expressions for strict email domain checks, 10-digit Indian phone numbers (6-9 prefix), and 8+ char complex passwords.
- **FileManager.java:** File I/O streams (BufferedReader/BufferedWriter) handling custom pipe serialization, timestamped audit logging, and HR CSV report exports.
- **Menu.java & Main.java:** 4-Portal Role-Based Console Architecture (Admin: admin_1234, Recruiter: Sarnine@123, Candidate, New Candidate Registration) with robust scanner exception handling.

3. Concept Summary Matrix

Java Concept / Principle	Target File(s)	Implementation Purpose
Abstraction	User.java	Abstract class template & abstract method displayDetails().
Inheritance	Candidate.java, Recruiter.java	Child models inheriting base user profile fields.
Encapsulation	All Models (src/model/)	Private attributes protected behind getters & setters.
Polymorphism (Overriding)	Candidate.java, Recruiter.java	Overriding displayDetails() and toString().
Polymorphism (Overloading)	CandidateService, JobService	Overloaded search methods (ID vs String queries).
Static Initializers	All Repositories (src/repo/)	Reading disk text files automatically on app startup.
File I/O Streams	FileManager.java	Disk persistence, audit logging, & CSV report export.
Regular Expressions (Regex)	Validation.java	Enforcing complex passwords, 6-9 phone prefixes, & emails.
Exception Handling	Menu.java	Catching InputMismatchException in input loops.

Role-Based Access Control	Menu.java	Password guarding Admin (admin_1234) & Recruiters.
---------------------------	-----------	--